ĐẠI HỌC PHENIKAA

TRƯỜNG CÔNG NGHỆ THÔNG TIN PHENIKAA

HỌC PHẦN: AN TOÀN PHẦN MỀM

ĐỀ TÀI: FUZZING WHITE BOX TESTING CHO QUẢN LÝ DÒNG HỌ

Giảng viên: ThS. Vũ Quang Dũng

Nhóm 5:

1. Nguyễn Tiến Lực (23010862 – K17 – KTPM(EL_2))

2. Quách Hữu Nam (23012358 – K17 – KTPM(EL_2))

3. Triệu Tiến Quỳnh (23010648 - K17 - KTPM(EL_2))

4. Lê Thị Kiều Trang (23010502 – K17 – KTPM(EL)_2)

Lớp học phần: CSE703093-1-1-26(N02)

Hà Nội, tháng 10 năm 2026

MỤC LỤC

1. Giới thiệu và mục tiêu 4
1.1. Bối cảnh và lý do lựa chọn đề tài 4
1.2. Mục tiêu cụ thể 4
1.3. Phạm vi và mức độ kết luận 5
1.4. Tóm tắt kết quả chính 5
2. Mô tả hệ thống và chương trình mục tiêu 5
2.1. Các thành phần của kho mã nguồn 5
2.2. Kiến trúc và luồng dữ liệu 6
2.3. Chức năng và dữ liệu trọng tâm 6
2.4. Quy mô mã nguồn 7
2.5. Bề mặt tấn công và ranh giới tin cậy 7
3. Phương pháp thực hiện 8
3.1. Mô hình hóa và thuộc tính an toàn 8
3.2. Một module trung tâm với năm bước xử lý 8
3.3. Trạng thái và điều kiện thực thi 9
3.4. Đặc tả 10 đường đi 10
3.5. Phân tích tĩnh và thực thi bằng sanitizer 11
3.6. Chiến dịch và các phép đo độ phủ 11
3.7. Kiểm thử bổ sung và phạm vi backend 12
4. Kết quả và phân tích 12
4.1. Kết quả chạy 11 ca kiểm thử gốc 12
4.2. Cảnh báo mô phỏng và sự cố thực tế 14
4.3. Lỗi C-01 còn tồn tại với độ dài audit âm 15
4.4. Kiểm thử biên và sai lệch mệnh đề 16

4.5. Kích thước vùng nhớ trên môi trường thực nghiệm 17
4.6. Vấn đề WEB-01 về phân quyền theo dòng họ 17
4.7. Vấn đề WEB-02 trong khởi tạo cơ sở dữ liệu 18
4.8. Giới hạn của bộ nhận dạng path 19
4.9. Đối chiếu README và bảng XLSX cập nhật 20
4.10. Tổng hợp mức độ bằng chứng 21
5. Thảo luận và hạn chế 21
5.1. Ý nghĩa của kết quả đối với an toàn phần mềm 21
5.2. Độ phủ và giới hạn soundness, completeness 22
5.3. Những giới hạn của thực nghiệm 22
5.4. Đề xuất khắc phục lỗi và hoàn thiện bộ kiểm thử 23
5.5. Hướng phát triển 24
6. Kết luận 24
Tài liệu tham khảo 26
Phụ lục 27
Phụ lục A. Tổ chức mã nguồn và bằng chứng 27
Phụ lục B. Chạy lại thực nghiệm 28
Phụ lục C. Đối chiếu kết luận với bằng chứng 28

1. Giới thiệu và mục tiêu

Link github dự án: https://github.com/lucnguyencoder/software_safety

1.1. Bối cảnh và lý do lựa chọn đề tài

Hệ thống quản lý dòng họ lưu trữ thông tin cá nhân, quan hệ gia đình, sự kiện và các khoản thu chi của dòng họ. Dữ liệu gia phả có quan hệ phụ thuộc giữa các thành viên; vì vậy, một thao tác cập nhật sai có thể ảnh hưởng đến nhiều nút trong cây. Đồng thời, việc tổ chức dữ liệu theo từng dòng họ đặt ra yêu cầu kiểm soát quyền truy cập đến từng bản ghi, bên cạnh việc kiểm tra tài khoản đã đăng nhập.

Kho mã nguồn software_safety-main-2 gồm ứng dụng web quản lý dòng họ và chương trình C clan_genealogy_fuzzer.c. Chương trình C xây dựng một module trung tâm gồm năm bước xử lý, 16 trạng thái, 16 nhãn mệnh đề, 10 đường đi được khai báo và 11 ca kiểm thử định sẵn. Đây là cơ sở để nghiên cứu cách thiết kế ca kiểm thử theo trạng thái, theo dõi độ phủ và đánh giá tính đúng đắn của cơ chế phát hiện lỗi.

Một vấn đề cần giải quyết là sự khác nhau giữa trạng thái được chương trình đặt tên là lỗi và lỗi thực tế được công cụ độc lập xác nhận. Nếu chỉ dựa vào dòng chữ “CRASH DETECTED” do chính chương trình in ra, người kiểm thử có thể kết luận sai về tính an toàn của mã nguồn. Do đó, báo cáo kết hợp đọc mã nguồn, thực thi mô hình và kiểm tra bằng AddressSanitizer để đánh giá bằng chứng ở từng mức.

1.2. Mục tiêu cụ thể

Báo cáo hướng đến các mục tiêu sau:

Mô tả kiến trúc, luồng dữ liệu, bề mặt tấn công và phạm vi kiểm thử của hệ thống.
Phân tích một module xử lý năm bước cùng quan hệ giữa trạng thái, điều kiện kiểm tra, vùng nhớ và đường đi thực thi.

Chạy lại 11 ca kiểm thử có sẵn; xác định độ phủ trạng thái, mệnh đề và đường đi; kiểm chứng ý nghĩa của các nhãn “CRASH”, “HANG”.

Bổ sung ca kiểm thử biên để phát hiện sai sót mà bộ kiểm thử ban đầu chưa xét đến.

Nhận diện một số vấn đề ở mã nguồn ứng dụng web, phân biệt kết luận phân tích tĩnh với kết quả kiểm thử tích hợp.

Đề xuất hướng khắc phục và cung cấp thông tin cần thiết để tái lập thực nghiệm.

1.3. Phạm vi và mức độ kết luận

Phần thực nghiệm động tập trung vào chương trình C và các hàm bên trong chương trình này. Phần ứng dụng web tập trung vào đường đi dữ liệu của chức năng thành viên gia phả, middleware xác thực và mã khởi tạo cơ sở dữ liệu. Báo cáo không khẳng định đã kiểm thử toàn bộ ứng dụng, không thực hiện tấn công vào hệ thống đang triển khai và không đánh giá hiệu năng với tải lớn.

Các kết quả được phân thành ba nhóm: kết quả thực thi đã quan sát, vấn đề suy ra từ mã nguồn và đề xuất chưa triển khai. Chương trình C là mô hình độc lập, không gọi trực tiếp API Express hay cơ sở dữ liệu MySQL. Vì vậy, lỗi trong mô hình C không tự động trở thành lỗi của ứng dụng web.

1.4. Tóm tắt kết quả chính

Bộ 11 ca có sẵn đạt 16/16 nhãn trạng thái, 16/16 nhãn mệnh đề và 10/10 đường đi khai báo. Việc đối chiếu độc lập toàn bộ 11 trace trong log với PATH_TABLE cho thấy các chuỗi khớp với đặc tả tương ứng. Chương trình tự ghi tám nhãn "CRASH" và một nhãn "HANG", nhưng kết thúc bình thường với mã 0 khi chạy cùng ASan/UBSan. Ca bổ sung có độ dài audit bằng -1 vẫn khiến ASan báo negative-size-param và dừng tiến trình.

Cấu trúc mới cho phép kết thúc sớm một ca khi gặp nhánh bất thường, nên trace phản ánh rõ vị trí dừng. Tuy nhiên, hàm ghi nhận path chỉ so khớp trạng thái cuối; tỷ lệ 100% chưa phải bằng chứng rằng mọi đường đi chương trình hoặc mọi miền đầu vào đã được kiểm thử. Báo cáo phân tích riêng kết quả quan sát và giới hạn của bộ đo này.

2. Mô tả hệ thống và chương trình mục tiêu

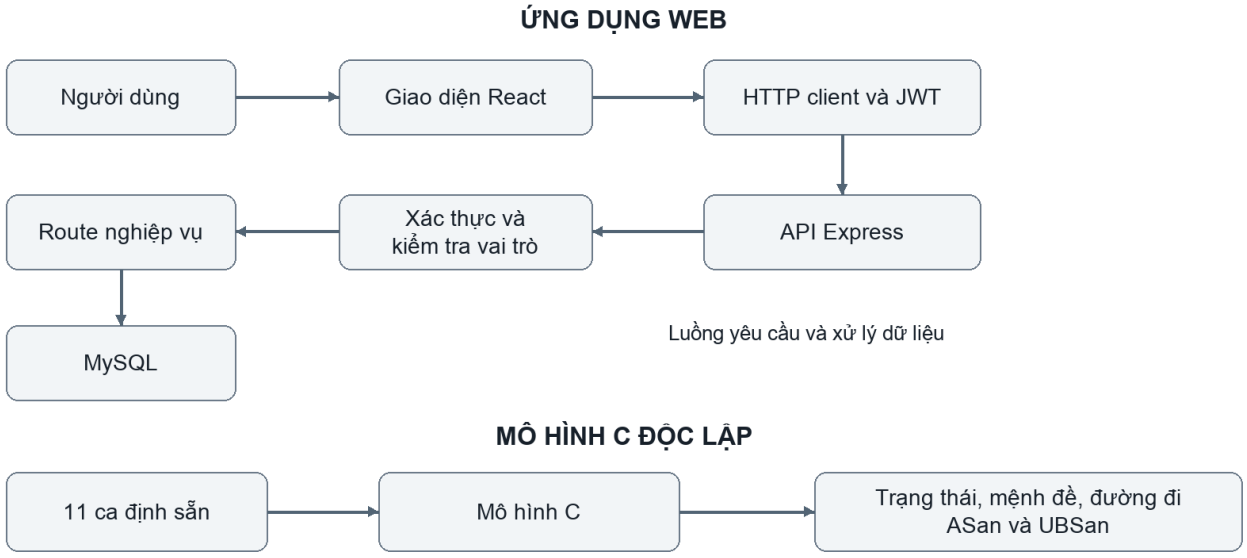
2.1. Các thành phần của kho mã nguồn

Thành phần	Công nghệ hoặc tệp chính	Vai trò
Giao diện web	React, TypeScript, Vite; quanly-dongho/src	Quản lý thành viên, hiển thị gia phả, sự kiện, tài khoản và tài chính
HTTP client	src/services/api.ts	Gửi yêu cầu JSON, đính kèm JWT trong header Authorization
Backend	Node.js, Express; backend/src	Xác thực, phân quyền, xử lý nghiệp vụ và truy vấn dữ liệu
Cơ sở dữ liệu	MySQL; backend/src/db	Lưu thành viên, tài khoản, sự kiện, giao dịch và nhật ký

Mô hình kiểm thử	clan_genealogy_fuzzer.c	Chạy các tình huống mô phỏng theo trạng thái và ghi log
Tài liệu đi kèm	README và bảng XLSX bốn sheet	Mô tả dự kiến về quy trình và kết quả; cần đối chiếu lại với code

Các phiên bản được khai báo trong package.json gồm React ^19.0.1, Vite ^6.2.3, Express backend ^4.19.2 và mysql2 ^3.10.1. Đây là khoảng phiên bản khai báo, không phải xác nhận phiên bản thư viện đã được cài đặt trong một hệ thống đang chạy.

2.2. Kiến trúc và luồng dữ liệu



Hình 2.2. Kiến trúc tổng quan. Nhánh ứng dụng web và nhánh mô hình C được thực thi độc lập. Liên hệ giữa hai nhánh nằm ở ý tưởng nghiệp vụ quản lý gia phả, chưa phải liên kết kiểm thử trực tiếp.

Ví dụ, khi người dùng cập nhật thành viên, giao diện gọi HTTP client, client gắn token rồi gửi yêu cầu đến API. Backend kiểm tra chữ ký token, trạng thái tài khoản và vai trò theo route. Dữ liệu được chuyển thành truy vấn SQL; kết quả trả lại dưới dạng JSON. Trong mô hình C, đầu vào được dựng sẵn trong cấu trúc FuzzTestCase, đi qua các hàm mô phỏng và không được lấy từ yêu cầu HTTP.

2.3. Chức năng và dữ liệu trọng tâm

Đối tượng thành viên có các trường họ tên, giới tính, tình trạng còn sống, ngày sinh, thế hệ, cha, mẹ và phối ngẫu. Backend cung cấp các thao tác liệt kê, xem chi tiết, thêm, sửa và xóa thông qua

/api/members. Các bảng khác phục vụ tài khoản, sự kiện, phản hồi tham dự, giao dịch quỹ và nhật ký thao tác.

Với mô hình C, mỗi ca kiểm thử có năm nhóm dữ liệu: tên và tiểu sử; chỉ số thành viên và thao tác vòng đời; chỉ số cha, phối ngẫu và cờ tạo chu trình; số bản ghi quỹ; độ dài sao chép nhật ký. Đầu ra trên stdout gồm log chi tiết, chuỗi trạng thái của từng ca, ba tỷ lệ độ phủ và các bộ đếm do chương trình tự tăng. Code hiện không tự mở tệp log; cần chuyển hướng stdout hoặc dùng công cụ chạy lại để lưu bằng chứng.

2.4. Quy mô mã nguồn

Số liệu được đếm theo số dòng vật lý, bao gồm dòng trống và chú thích, không phải số dòng lệnh thực thi.

Phạm vi	Số tệp	Số dòng
clan_genealogy_fuzzer.c	1	532
quanly-dongho/src với đuôi mã nguồn được thống kê	28	15.189
quanly-dongho/backend/src	14	2.639
Tổng các tệp .c, .js, .ts, .tsx, .css, .html trong kho	45	18.395

Tổng toàn kho còn gồm các tệp ở ngoài hai thư mục src, nên không bằng tổng ba hàng đầu. Tệp C có 11 hàm được định nghĩa. Hàm execute_clan_genealogy_module là điểm thực thi nghiệp vụ trung tâm; hai hàm duyệt xử lý đồ thị, các hàm còn lại phục vụ log, đo độ phủ, tạo chiến dịch và điều phối. Module_Clan_Genealogy_Engine là tên của một module logic, không phải năm module như phiên bản trước.

2.5. Bề mặt tấn công và ranh giới tin cậy

Điểm nhận dữ liệu	Dữ liệu cần quan tâm	Thuộc tính cần bảo vệ
Đăng nhập, đăng ký	Định danh, mật khẩu, mã mời	Xác thực, chống lạm dụng, bảo vệ thông tin tài khoản
Route thành viên	:id, bộ lọc, thông tin cá nhân, quan hệ gia đình	Quyền trên bản ghi, toàn vẹn quan hệ và dữ liệu
Route tài chính	Số tiền, loại giao dịch, năm định mức	Tính hợp lệ của giá trị và quyền ghi dữ liệu

Route tư liệu sự kiện	Tệp tải lên, thông tin sự kiện	Quyền truy cập và giới hạn dữ liệu
Hàm C qua chương trình kiểm thử bổ sung	Độ dài, chỉ số, chuỗi, số lượng	An toàn bộ nhớ và tính đúng của điều kiện kiểm tra

JWT và kiểm tra vai trò là các bước cần thiết, nhưng chưa đủ để bảo đảm người dùng chỉ truy cập bản ghi thuộc dòng họ được phép. Tương tự, dữ liệu đã qua kiểm tra giao diện vẫn cần được xác thực tại backend vì yêu cầu HTTP có thể được tạo ngoài giao diện.

3. Phương pháp thực hiện

3.1. Mô hình hóa và thuộc tính an toàn

Chương trình được xem xét qua đầu vào, trạng thái trung gian, điều kiện chuyển tiếp và đường đi đến trạng thái kết thúc. Các thuộc tính cần kiểm tra gồm: độ dài sao chép không âm và không vượt kích thước nguồn/đích; chỉ số trong miền hợp lệ; con trỏ còn hợp lệ khi sử dụng; kích thước không mất dữ liệu trước khi sử dụng; phép duyệt dừng khi có chu trình; thao tác web được giới hạn đúng dòng họ.

Các thuộc tính này định hướng việc đọc code và thiết kế kiểm thử. Báo cáo không áp dụng SMT/BMC, không thực hiện symbolic execution và không tuyên bố đã chứng minh hình thức chương trình an toàn.

3.2. Một module trung tâm với năm bước xử lý

Module Module_Clan_Genealogy_Engine được thực thi bởi execute_clan_genealogy_module. Tại đầu mỗi ca, code xóa trace hiện tại và bộ đếm trạng thái/path mới; các mảng độ phủ toàn chiến dịch vẫn được giữ lại.

Bước	Vai trò	Vùng nhớ và hành vi chính
1 Parser	Kiểm tra tên và tiểu sử	name_buf[16], bio_buf[64]; kiểm tra độ dài và %x/%n
2 Heap Node	Cấp phát và phân loại vòng đời nút	g_member_table[16], malloc(sizeof(ClanMember)), nhánh giải phóng rồi cấp phát lại
3 Linker	Phân loại quan hệ	Kiểm tra chỉ số cha; kiểm tra đặc biệt ô phối ngẫu số 15
4 DFS BFS và Fund	Duyệt đồ thị và tính kích thước	g_parent_map, hàng đợi 64 số nguyên, phép thu hẹp về uint16_t

5 Audit Diff	Sao chép dữ liệu đối chiếu	old_bio_buf[24] và diff_preview[64]

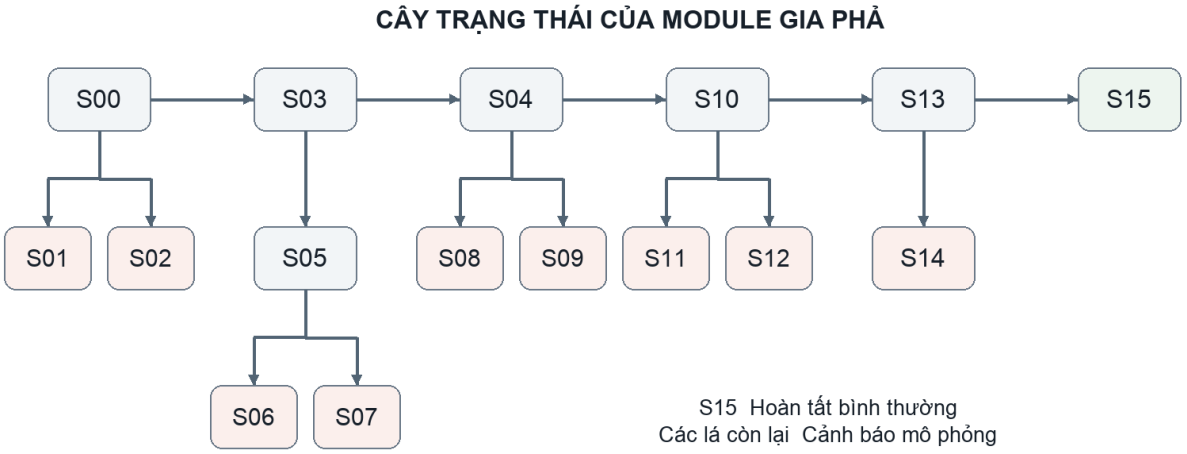
Khác với cách chạy tiếp qua các module của phiên bản trước, các nhánh bất thường hiện gọi goto finalize_path. Một ca chỉ đi tiếp nếu bước trước đạt nhánh bình thường. Sau khi kết thúc đường đi, chương trình ghi nhận trace và in độ phủ, rồi tiếp tục ca tiếp theo. “Kết thúc sớm một ca” không có nghĩa tiến trình bị crash.

3.3. Trạng thái và điều kiện thực thi

Bảng dưới đây diễn giải theo code; tên trạng thái được giữ để đối chiếu với STATE_TABLE. Mỗi trạng thái đi kèm một nhãn mệnh đề cùng số thứ tự.

Trạng thái	Mệnh đề	Điều kiện hoặc hành động trong code
S00	P00	Vào module sau khi in đầu vào; không có phép kiểm tra NULL riêng
S01	P01	Tên dài từ 16 byte; chép tối đa 15 byte và kết thúc ca
S02	P02	Tên ngắn; tiểu sử chứa %x hoặc %n; dùng định dạng %s rồi kết thúc ca
S03	P03	Tên ngắn và tiểu sử không chứa hai mẫu %x, %n
S04	P04	Đến nhánh lifecycle_op <= 0; nút đã có hoặc vừa được cấp phát
S05	P05	lifecycle_op > 0; đã gọi free, con trỏ chưa được gán lại
S06	P06	lifecycle_op == 1; chỉ ghi nhãn UAF
S07	P07	lifecycle_op > 0 và khác 1; không chỉ riêng giá trị 2
S08	P08	Chỉ số cha ngoài 0–15; không thực hiện ghi mảng qua chỉ số đó
S09	P09	Chỉ số phối ngẫu bằng 15 và ô 15 là NULL; không giải tham chiếu NULL
S10	P10	Chỉ số cha hợp lệ và không vào nhánh kiểm tra đặc biệt ở S09
S11	P11	Ít nhất một phép duyệt vượt 32 bước rồi trả -1
S12	P12	Không có chu trình; fund_count > 16384 và số byte thu hẹp nhỏ hơn số bản ghi
S13	P13	Không vào hai nhánh cảnh báo của bước 4
S14	P14	Độ dài audit lớn hơn 24; kết thúc ca trước memcpy

S15	P15	Nhánh còn lại của audit, ghi sau memcpy; còn thiếu kiểm tra độ dài âm



Hình 3.3. Cây chuyển trạng thái của module. Các lá S01, S02, S06, S07, S08, S09, S11, S12, S14 và S15 tương ứng với 10 đường đi khai báo. S15 là lá hoàn tất bình thường; các lá khác mang nhãn cảnh báo mô phỏng.

3.4. Đặc tả 10 đường đi

Path	Chuỗi trạng thái khai báo	Ca kích hoạt
PATH_01	S00 → S03 → S04 → S10 → S13 → S15	TC01, TC02
PATH_02	S00 → S01	TC03
PATH_03	S00 → S02	TC04
PATH_04	S00 → S03 → S04 → S08	TC05
PATH_05	S00 → S03 → S04 → S09	TC06
PATH_06	S00 → S03 → S04 → S10 → S12	TC07
PATH_07	S00 → S03 → S04 → S10 → S13 → S14	TC08
PATH_08	S00 → S03 → S05 → S06	TC09
PATH_09	S00 → S03 → S05 → S07	TC10
PATH_10	S00 → S03 → S04 → S10 → S11	TC11

g_current_trace chứa tối đa tám trạng thái; đường đi dài nhất đang khai báo có sáu trạng thái. Trace lưu các mốc trong mô hình, không lưu từng bước lặp DFS/BFS, từng nhánh malloc hoặc mọi nhánh điều khiển của ngôn ngữ C. Vì vậy, 10 path là 10 chuỗi được mô hình hóa, không phải toàn bộ đường đi có thể có của chương trình.

3.5. Phân tích tĩnh và thực thi bằng sanitizer

Quá trình kiểm tra đối chiếu ba lớp: điều kiện thực sự chạy, thao tác bộ nhớ và cơ chế sinh kết quả. README và bốn sheet XLSX được dùng làm tài liệu tham khảo, nhưng kết luận dựa vào code cùng kết quả chạy lại. Quy mô mã nguồn được đếm theo dòng vật lý; kích thước bộ nhớ được đo bằng sizeof trên môi trường thực nghiệm.

AddressSanitizer được chọn để xác nhận thao tác bộ nhớ không hợp lệ trong các đường đi đã thực thi. UndefinedBehaviorSanitizer được bật cùng để bổ sung kiểm tra các hành vi không xác định được công cụ hỗ trợ [3], [4].

Lệnh biên dịch:

```
clang -std=c11 -Wall -Wextra -g -O0 \
  -fsanitize=address,undefined -fno-omit-frame-pointer \
  source/clan_genealogy_fuzzer.c -o results/baseline
```

Môi trường kiểm tra ngày 23/10/2026 là macOS 15.6.1, ARM64, Apple Clang 17.0.0. -O0 giảm việc tối ưu bỏ thao tác phục vụ kiểm thử; -g và -fno-omit-frame-pointer hỗ trợ lần theo lỗi. Khi chạy, ASAN_OPTIONS=detect_leaks=0 được sử dụng nên không đánh giá memory leak. Mỗi tiến trình có giới hạn 45 giây; ca lỗi độ dài âm bị sanitizer chấm dứt trước giới hạn này.

3.6. Chiến dịch và các phép đo độ phủ

Chiến dịch gồm hai seed, sáu biến thể BFS_QUEUE và ba biến thể DFS_STACK. Các biến thể được gán cố định trong code rồi chạy tuần tự. Không có bộ sinh ngẫu nhiên, giải ràng buộc hay cơ chế chọn đầu vào mới theo phản hồi độ phủ.

Hai hàm duyệt đồ thị vẫn có hoạt động cụ thể: traverse_depth_dfs đi theo chuỗi cha bằng vòng lặp; traverse_children_bfs dùng hàng đợi, xét bốn nút đầu. Các tên BFS/DFS của mảng ca kiểm thử mô tả cách tổ chức các tình huống, chưa chứng minh có thuật toán tự động khám phá không gian trạng thái. Báo cáo vì vậy xem phiên bản này là kiểm thử định sẵn dựa trên cấu trúc mã nguồn và trạng thái, chưa phải một chiến dịch coverage-guided fuzzing hoặc symbolic white-box fuzzing hoàn chỉnh.

State Coverage = số phần tử được đánh dấu trong g_state_visited / 16. Predicate Coverage được tính tương tự với g_predicate_visited. enter_state đánh dấu hai loại nhãn cùng lúc; không có bộ đánh giá độc lập tính lại biểu thức mệnh đề. Path Coverage = số phần tử đã đánh dấu trong g_path_visited / 10. Hàm record_execution_path hiện chỉ so sánh trạng thái cuối với terminal_state của từng path.

Để kiểm tra ý nghĩa của số đo path, báo cáo bổ sung hai việc: so sánh toàn bộ trace quan sát trong baseline với chuỗi khai báo và kiểm thử riêng bộ nhận dạng bằng một trace không khớp nhưng có cùng trạng thái cuối. Hai việc này kiểm tra các khía cạnh khác nhau: tính đúng của dữ liệu lần chạy hiện tại và khả năng phân biệt của bộ đo.

3.7. Kiểm thử bổ sung và phạm vi backend

tools/probe.c đổi tên main khi include tệp C gốc, tạo một ca nền hợp lệ rồi gọi execute_clan_ genealogy_module. Các nhóm đầu vào gồm audit -1, 0, 24, 25; fund 16384, 16385, 32768; tiểu sử 100% dong gop; chỉ số phối ngẫu 14; thao tác vòng đời 3; và đo sizeof. Mỗi lệnh chạy trong một tiến trình mới, sau đó giải phóng các nút nếu tiến trình hoàn tất.

Một kiểm thử đơn vị riêng gán trace S00 → S15 rồi gọi record_execution_path để kiểm tra cơ chế so khớp. Trace này được dựng chủ động cho bộ đo; không được báo cáo như một đường đi thực thi tự nhiên của module.

Các tệp frontend/backend không thay đổi so với bản trước, được xác định bằng đối chiếu nội dung. Nhận xét phân quyền vì vậy được giữ và kiểm tra lại vị trí mã. Kiểm thử runSchema với kết nối giả được chạy lại, không kết nối MySQL và không xác nhận ứng dụng web đã chạy tích hợp.

4. Kết quả và phân tích

4.1. Kết quả chạy 11 ca kiểm thử gốc

Bảng dưới đây ghi kết quả thực thi của bản cập nhật. Các tỷ lệ State/Pred trùng nhau vì chương trình đánh dấu hai loại nhãn tại cùng lời gọi enter_state.

Ca	Đầu vào hoặc thay đổi chính	Trạng thái mới	State/Pred	Path
TC01	Seed gốc; tên 13 byte; audit 12	S00, S03, S04, S10, S13, S15	37,5%	10%; PATH_01
TC02	Seed nhánh; slot 1; audit 18	Không	37,5%	10%; PATH_01

TC03	Tên dài 44 byte	S01	43,8%	20%; PATH_02
TC04	Tiểu sử chứa %x, %n	S02	50,0%	30%; PATH_03
TC05	parent_idx = -5	S08	56,2%	40%; PATH_04
TC06	spouse_idx = 15	S09	62,5%	50%; PATH_05
TC07	fund_count = 16385	S12	68,8%	60%; PATH_06
TC08	audit_claim_len = 60	S14	75,0%	70%; PATH_07
TC09	lifecycle_op = 1	S05, S06	87,5%	80%; PATH_08
TC10	lifecycle_op = 2	S07	93,8%	90%; PATH_09
TC11	create_cycle = 1	S11	100,0%	100%; PATH_10

Bảng 4.1. Bảng kết quả của chiến dịch. Cột Path ghi tỷ lệ tích lũy và mã path của ca hiện tại. Tỷ lệ được giữ theo cách làm tròn một chữ số thập phân của log; 9/16 được in là 56,2% trên môi trường thực nghiệm.

Nguồn: results/baseline.stdout.txt.

```
Total Test Cases Executed: 11 (2 Seeds + 6 BFS + 3 DFS)
State Coverage:     100.0% (16 / 16 States)
Predicate Coverage: 100.0% (16 / 16 Predicates)
Path Coverage:      100.0% (10 / 10 Execution Paths)
Bo dem CRASH do chuong trinh tang: 8
Bo dem HANG do chuong trinh tang:  1
Ma thoat thuc te cua baseline:     0
Chan doan ASan/UBSan trong stderr: khong co
```

Tên dài của TC03 là Nguyen_Van_Truong_Toc_Dai_Chi_Stack_Overflow, có strlen = 44 byte; vùng lưu chuỗi gồm ký tự kết thúc cần ít nhất 45 byte. Hai seed đều đi đến PATH_01; chín biến thể mở thêm chín lá khác. Đối chiếu độc lập toàn bộ 11 trace với PATH_TABLE xác nhận 11/11 trace khớp, bao phủ đủ 10 chuỗi khai báo trong lần chạy này.

4.2. Cảnh báo mô phỏng và sự cố thực tế

Hàm enter_state tăng bộ đếm dựa trên chuỗi outcome_type, không dựa trên tín hiệu hệ điều hành hoặc thông báo từ sanitizer. Nhánh goto finalize_path dừng đường đi hiện tại nhưng không dừng tiến trình. Vì vậy, tám nhãn CRASH và một nhãn HANG chưa thể được gọi là tám crash thật và một lần treo thật.

Nhãn và CWE trong mô hình	Hành vi thực tế	Kết luận quan sát
S01, CWE-121	Chỉ chép 15 byte vào tên 16 byte rồi thêm ký tự kết thúc	Không tái hiện ghi tràn bộ đệm
S02, CWE-134	Dùng định dạng cố định %s với dữ liệu tiểu sử	Không dùng đầu vào làm format string
S06, CWE-416	Ghi nhãn sau free rồi cấp phát lại	Không đọc/ghi qua vùng đã giải phóng ở nhánh này
S07, CWE-415	Chỉ một lời gọi free trước khi cấp phát lại	Không thực hiện double free
S08, CWE-787	Kiểm tra chỉ số cha rồi kết thúc ca	Không ghi ngoài mảng qua chỉ số đó
S09, CWE-476	Kiểm tra ô 15 bằng NULL rồi kết thúc ca	Không giải tham chiếu con trỏ NULL
S11, CWE-674	DFS/BFS dừng sau 33 bước và trả -1	Phát hiện vượt ngưỡng, không có treo thực tế
S12, CWE-190	Thu hẹp kích thước rồi ghi nhãn; không cấp phát quỹ	Chưa có heap overflow được thực hiện
S14, CWE-125	Kết thúc ca trước memcpy khi độ dài > 24	Nhánh này không đọc vượt biên

Bảng 4.2. Bảng đối chiếu cảnh báo với thao tác thực thi. Các mã CWE ở cột đầu được lấy từ bảng mô hình, không phải danh sách lỗ hổng đã xác nhận trên ứng dụng web.

Các hàm duyệt của C dùng vòng lặp có giới hạn, không dùng đệ quy. Nhãn CWE-674 do đó không phản ánh trực tiếp cách triển khai. Nếu bỏ chặn và để chu trình gây vòng lặp không dừng, CWE-835

gần với cơ chế đó hơn [8]; hiện tại log TC11 chỉ chứng minh DFS/BFS đạt 33 bước rồi kết thúc đường đi.

4.3. Lỗi C-01 còn tồn tại với độ dài audit âm

Vị trí cập nhật: clan_genealogy_fuzzer.c, hàm execute_clan_genealogy_module, kiểm tra ở dòng 366 và memcpy ở dòng 371. Hàm audit riêng của phiên bản cũ không còn tồn tại.

```
memset(diff_preview, 0, sizeof(diff_preview));
if (tc->step5_audit_claim_len > (int)sizeof(old_bio_buf)) {
    enter_state(S14_ERR_AUDIT_OOB_READ, 14, tc);
    goto finalize_path;
}
memcpy(diff_preview, old_bio_buf,
       (size_t)tc->step5_audit_claim_len);
enter_state(S15_AUDIT_COMMITTED_OK, 15, tc);
```

Đầu vào tái hiện: ca nền có tên Nguyen Van A, tiểu sử Tieu su, slot/cha/phối ngẫu đều bằng 0, không tạo chu trình, fund_count = 10; chỉ thay step5_audit_claim_len = -1. Các bước trước audit đi theo nhánh bình thường.

Điều kiện -1 > 24 sai nên không vào S14. Giá trị âm sau đó được đổi sang size_t, trở thành số nguyên không dấu rất lớn và được dùng làm kích thước memcpy. ASan dừng tiến trình trước khi module ghi S15 hoặc hoàn tất đo path của ca này.

Bằng chứng rút gọn từ results/audit_-1.stderr.txt:

```
ERROR: AddressSanitizer: negative-size-param: (size=-1)
    ... in __asan_memcpy
    ... in execute_clan_genealogy_module
    ... in main
SUMMARY: AddressSanitizer: negative-size-param
ABORTING
```

Python ghi nhận returncode = -6, tương ứng tín hiệu 6 trên môi trường thực nghiệm. Log có cảnh báo về trình giải ký hiệu bên ngoài, nhưng vẫn nêu rõ loại lỗi và hàm liên quan. Kết quả xác nhận tham số kích thước sai và tiến trình bị dừng; không chứng minh đã ghi đè thành công bộ nhớ hay có khai thác từ mạng.

Nguyên nhân phù hợp với CWE-195, lỗi chuyển số có dấu sang số không dấu [6]. Cần ưu tiên sửa trước khi dùng hàm với đầu vào biến đổi. main gốc chưa tạo số âm và mô hình C không được nối vào API web, nên phạm vi đã xác nhận vẫn là ca bổ sung gọi module C.

4.4. Kiểm thử biên và sai lệch mệnh đề

Thử nghiệm	Quan sát trên bản cập nhật	Ý nghĩa
Audit = -1	ASan báo lỗi; tiến trình bị chấm dứt	C-01 vẫn tồn tại
Audit = 0 hoặc 24	S15, PATH_01, mã thoát 0	Không có chẩn đoán ở hai biên được kiểm tra
Audit = 25	S14, PATH_07, mã thoát 0	Cảnh báo mô hình; không có đọc ngoài biên
Fund = 16384	Cần 65536 byte, thu hẹp thành 0; S12 không được ghi	Bộ phát hiện bỏ sót biên đầu tiên mất dữ liệu
Fund = 16385	Cần 65540 byte, thu hẹp thành 4; S12	Được phát hiện theo điều kiện đã cài
Fund = 32768	Cần 131072 byte, thu hẹp thành 0; S12	Một giá trị lớn hơn được phát hiện
Tiểu sử 100% dong gop	S03 rồi PATH_01	P03 không thực sự yêu cầu không có mọi dấu %
Phối ngẫu = 14; ô 14 là NULL	S10 rồi PATH_01	Không kiểm tra tổng quát con trỏ phối ngẫu như mô tả P10
Vòng đời = 3	S05 rồi S07, PATH_09	S07 được gán cả khi giá trị khác 2

Bảng 4.4. Bảng kiểm thử bổ sung. Ngoài audit bằng -1, các tiến trình trên kết thúc với mã 0. PATH_01 ở một ca bất thường không chứng minh các điều kiện nghiệp vụ đã đúng.

Phép tính kích thước ở dòng 356–357 còn điều kiện:

```
uint16_t alloc_bytes =
    (uint16_t)(tc->step4_fund_count * sizeof(uint32_t));
if (tc->step4_fund_count > 16384 &&
    alloc_bytes < tc->step4_fund_count) {
    enter_state(S12_ERR_FUND_INT_OVERFLOW, 12, tc);
    goto finalize_path;
}
```

Ở count 16384, kết quả toán học 65536 bị thu hẹp thành 0, nhưng điều kiện > 16384 sai. Đây là lỗi của bộ phát hiện (C-02). So sánh số byte sau thu hẹp với số bản ghi cũng không trực tiếp kiểm tra kích thước có được bảo toàn. CWE-197 mô tả việc mất dữ liệu khi đổi sang kiểu nhỏ hơn [7]. Phép đổi sang kiểu nguyên không dấu nhỏ hơn không mặc nhiên là undefined behavior, nên không có cảnh báo UBSan ở đây không chứng minh phép tính đúng mục đích.

Các kiểm thử tiểu sử, phối ngẫu và vòng đời còn chứng minh biểu thức trong STATE_TABLE chưa đồng nhất với điều kiện chạy. Ví dụ, P10 ghi con trỏ phối ngẫu khác NULL, nhưng code chỉ kiểm tra ô 15; P07 ghi thao tác bằng 2, nhưng nhánh thực chấp nhận mọi số dương khác 1. Việc đánh dấu nhãn mệnh đề không thay thế được kiểm tra biểu thức.

4.5. Kích thước vùng nhớ trên môi trường thực nghiệm

```
sizeof(ClanMember)=72 sizeof(void*)=8 sizeof(int)=4
```

Cấu trúc mới đã bỏ hai trường liên kết so với bản trước; phép đo hiện tại là 72 byte trên ARM64, không dùng lại con số 80 byte của lần phân tích cũ. Mô tả “64 bytes” trong STATE_TABLE, README và dữ liệu tham khảo vẫn chưa khớp với phép đo này. Chênh lệch xuất phát từ các trường dữ liệu, con trỏ và phần đệm căn chỉnh; cần đo lại khi đổi ABI.

g_member_table[16] chiếm 128 byte với con trỏ 8 byte. Hàng đợi BFS có 64 số nguyên, chiếm 256 byte với sizeof(int) = 4. Mô tả hàng đợi [64] trong bảng trạng thái mới đã phù hợp với khai báo. Các bộ đệm cục bộ gồm tên 16 byte, tiểu sử 64 byte, nguồn audit 24 byte và đích audit 64 byte. Dòng mô tả “Heap Fund Buffer” chỉ là mô phỏng, vì code không có cấp phát vùng quỹ trong bước 4.

4.6. Vấn đề WEB-01 về phân quyền theo dòng họ

Mức bằng chứng: phân tích tĩnh; chưa thử nghiệm HTTP tích hợp.

Trong backend/src/routes/members.js, route danh sách sử dụng điều kiện clanId = ? lấy từ req.user.clanId. Tuy nhiên, route xem chi tiết ở dòng 55–65 truy vấn thành viên chỉ theo id. Các thao tác sửa và xóa cũng có truy vấn dựa trên id mà không đồng thời ràng buộc clanId của người dùng.

Đoạn mã tiêu biểu của route chi tiết:

```
router.get("/:id", async (req, res) => {
  try {
    const [rows] = await pool.query(
      "SELECT * FROM tbl_members WHERE id = ?",
      [req.params.id]
    );
    if (!rows.length) {
      return res.status(404).json({ error: "Không tìm thấy thành viên." });
    }
    return res.json(rows[0]);
  } catch (err) {
    return res.status(500).json({ error: "Lỗi server." });
  }
});
```

Middleware chung yêu cầu đăng nhập và trạng thái ACTIVE. Với thao tác sửa, route còn yêu cầu vai trò LEADER. Tuy nhiên, những kiểm tra đó không ràng buộc bản ghi được chọn với dòng họ của tài khoản. Nếu một hệ thống triển khai lưu nhiều dòng họ và có dữ liệu tương ứng, việc thay đổi định danh trong yêu cầu có nguy cơ cho phép truy cập chéo phạm vi. Cơ chế này phù hợp với nhóm CWE-639 [9].

Tác động tiềm tàng: lộ thông tin cá nhân hoặc sửa, xóa thành viên của dòng họ khác tùy quyền route. Mức ưu tiên khắc phục cao trong mô hình nhiều dòng họ, nhưng chưa có bằng chứng khai thác trên một phiên bản đã triển khai. Cần kiểm tra tích hợp bằng hai tài khoản thuộc hai dòng họ riêng biệt trước khi khẳng định phạm vi tác động thực tế.

4.7. Vấn đề WEB-02 trong khởi tạo cơ sở dữ liệu

Mức bằng chứng: đọc mã nguồn và kiểm thử đơn vị với cơ sở dữ liệu giả.

Trong backend/src/db/schema.js, hàm runSchema tách chuỗi SQL theo dấu chấm phẩy rồi loại mọi phần bắt đầu bằng --:

```
const statements = SCHEMA_SQL
  .split(";")
  .map(s => s.trim())
  .filter(s => s.length > 0 && !s.startsWith("--"));

for (const sql of statements) {
  await conn.query(sql);
}
console.log("Schema created/verified");
```

Các lệnh tạo bảng chính trong chuỗi đều được đặt sau chú thích mở đầu --. Vì chú thích và câu SQL cùng nằm trong một phần sau khi tách, bộ lọc loại cả phần đó. Kiểm thử thực thi hàm gốc với kết nối giả ghi nhận:

```
{
  "test": "runSchema with mock DB",
  "base_statements_sent": 0,
  "queries": []
}
```

Dù không gửi câu lệnh tạo bảng chính nào, hàm vẫn in thông báo đã tạo hoặc kiểm tra schema. Kết quả không có nghĩa toàn bộ tệp schema.js không gửi SQL: một số bảng bổ sung và migration được gọi riêng ngoài runSchema. Vấn đề được xác nhận nằm ở danh sách câu lệnh chính của hàm này.

Ngoài ra, phần định nghĩa bảng tài khoản và thành viên trong tệp schema chưa có cột clanId, trong khi các route có sử dụng cột này. Đây là điểm cần đối chiếu với migration hoặc schema thực tế khi dựng lại ứng dụng. Báo cáo chưa chạy MySQL để kiểm tra trạng thái một cơ sở dữ liệu có sẵn.

Tác động: trên cơ sở dữ liệu mới, các bảng chính có nguy cơ không được tạo như dự kiến, ảnh hưởng trực tiếp đến khả năng cài đặt và chạy lại. Đây là lỗi triển khai và tính tái lập đã có bằng chứng ở cấp hàm; không gán một mức CVSS hay mã CWE không cần thiết.

4.8. Giới hạn của bộ nhận dạng path

Mã phát hiện C-04; mức bằng chứng: phân tích tĩnh, kiểm thử đơn vị bộ đo và đối chiếu trace baseline.

Mặc dù chú thích hàm nói đối chiếu chuỗi trạng thái, phần quyết định trong record_execution_path chỉ sử dụng trạng thái cuối:

```
SystemStateID terminal =
  g_current_trace[g_current_trace_len - 1];
    for (int p = 0; p < TOTAL_PATHS; p++) {
        if (PATH_TABLE[p].terminal_state == terminal) {
       g_path_visited[p] = 1;
       /* Ghi nhan path va dung tim kiem. */
       break;
    }
 }
```

Đoạn trên rút gọn phần log và bộ đếm, giữ nguyên tiêu chí so khớp. state_sequence trong bảng path là chuỗi dùng để mô tả/in báo cáo, không được so sánh với trace để quyết định path đã phủ.

Kiểm thử đơn vị dựng trace S00 → S15, ngắn hơn PATH_01, rồi gọi bộ nhận dạng. Kết quả:

```
[PATH COVERAGE TRACE] S00 -> S15
Matched: PATH_01 (Full_Normal_Commit_Path)
PATH_01_visited=1
```

Kết quả này chứng minh bộ nhận dạng không kiểm tra toàn bộ chuỗi. Nó không chứng minh trace rút gọn là một đường đi tự nhiên có thể đạt từ module hiện tại. Với cây trạng thái đang có, mỗi path có trạng thái cuối khác nhau và 11 trace baseline thực tế đều khớp chuỗi khai báo. Vì vậy, số 10/10 của lần chạy hiện tại có thêm bằng chứng từ việc đối chiếu trace, nhưng thuật toán đo vẫn cần được sửa nếu muốn xác thực đường đi đầy đủ hoặc mở rộng mô hình.

Độ phủ còn bỏ qua số lần lặp DFS/BFS, kết quả kiểm tra g_member_table[slot] == NULL, các giá trị khác nhau của từng miền số và các chuỗi cùng đi vào một nhãn. Không nên diễn giải 100% path là đã kiểm thử mọi đường điều khiển của C.

4.9. Đối chiếu README và bảng XLSX cập nhật

README hiện hướng dẫn biên dịch GCC và chạy trực tiếp, mô tả đúng quy mô một module, 16 trạng thái, 16 nhãn mệnh đề, 10 path và 11 ca. Hướng dẫn libFuzzer cùng mô tả 50.000 lượt của bản cũ đã được bỏ, nên không được tiếp tục ghi trong báo cáo như lỗi của README hiện tại.

Bảng XLSX mới có bốn sheet: trạng thái/mệnh đề/vùng nhớ, đặc tả 10 path, 11 ca kiểm thử và log. Cấu trúc và các tổng số phù hợp với phiên bản C mới. Tuy nhiên, các nhãn “CRASH”, “HANG” và kích

thước heap 64 byte trong tài liệu vẫn phải được đọc theo giới hạn đã phân tích, không thay thế bằng chứng sanitizer và phép đo sizeof.

Tệp C chưa có entry point LLVMFuzzerTestOneInput; báo cáo không chạy hoặc tuyên bố có chiến dịch libFuzzer. Việc phát triển harness theo giao diện chuẩn [5] chỉ được nêu như hướng cải thiện. Log gốc được gửi ra stdout, vì vậy bằng chứng của lần chạy này nằm trong baseline.stdout.txt, không phải tệp do chương trình tự mở.

4.10. Tổng hợp mức độ bằng chứng

Mã	Phát hiện	Bằng chứng	Phạm vi kết luận
C-01	Độ dài audit âm đến memcpy	ASan và mã kết thúc -6	Lỗi xác nhận trong module C
C-02	Bỏ sót count = 16384	Giá trị trước/sau thu hẹp, không có S12	Sai sót của điều kiện phát hiện
C-03	Nhãn và mô tả chưa khớp hành vi	Baseline, sizeof, các probe về mệnh đề	Tám nhãn CRASH chưa là tám crash thực
C-04	Path chỉ được nhận dạng bằng trạng thái cuối	Code và trace đơn vị S00 → S15	Giới hạn của bộ đo, không phải đường đi tự nhiên đã xác nhận
WEB-01	Truy vấn theo id thiếu phạm vi dòng họ	Route và middleware không thay đổi	Nguy cơ phân quyền cần kiểm thử tích hợp
WEB-02	runSchema bỏ các câu tạo bảng chính	Hàm gốc chạy với kết nối giả	Lỗi được tái hiện ở cấp hàm

5. Thảo luận và hạn chế

5.1. Ý nghĩa của kết quả đối với an toàn phần mềm

Thực nghiệm cho thấy ba lớp đánh giá cần được tách biệt: đạt nhãn trạng thái/path trong mô hình, thực thi hành vi nguy hiểm và tác động lên hệ thống triển khai. Bộ ca gốc làm tốt việc đi qua các nhãn đã được thiết kế. Tuy nhiên, nhiều nhãn chỉ biểu diễn tình huống giả định; chưa có thao tác bộ nhớ tương ứng để tạo lỗi thực. Vì vậy, bộ đếm phản ánh hoạt động của mô hình hơn là số lượng lỗ hổng được công cụ xác nhận.

Trường hợp độ dài âm minh họa lợi ích của kết hợp đọc code và kiểm thử động. Phân tích điều kiện cho thấy chỉ có cận trên; ca kiểm thử -1 biến nhận xét đó thành bằng chứng thực thi. Ngược lại, chỉ đọc log mô phỏng sẽ bỏ qua lỗi này vì cả S14 và S15 đã được đánh dấu sau bộ ca ban đầu.

Đối với ứng dụng web, lỗi bộ nhớ của C không thay thế được việc kiểm tra quyền truy cập dữ liệu. Rủi ro quan trọng có thể xuất hiện tại ranh giới nghiệp vụ, chẳng hạn quyền truy cập bản ghi của dòng họ khác, dù không liên quan đến tràn bộ đệm.

5.2. Độ phủ và giới hạn soundness, completeness

Tỷ lệ 100% trong báo cáo là độ phủ 16 nhãn trạng thái, 16 nhãn mệnh đề và 10 path đã khai báo. Nó không phải độ phủ toàn bộ câu lệnh, nhánh, đường đi, miền số nguyên hay API của ứng dụng. Mỗi trạng thái có thể bao gồm nhiều đầu vào với hành vi khác nhau: nhánh còn lại sau kiểm tra claim_len > 24 bao gồm cả số âm và các giá trị không âm hợp lệ.

Nếu hiểu nhãn “CRASH” là kết luận có crash thực, cơ chế hiện tại tạo ra nhiều cảnh báo không tương ứng với sự cố đã quan sát. Nếu hiểu việc đạt 100% là đã tìm hết lỗi, trường hợp -1 là phản ví dụ cho kết luận đó. Đây là liên hệ thực tế với yêu cầu xem xét tính đúng và tính đầy đủ của công cụ phân tích, nhưng báo cáo không tuyên bố đã chứng minh hình thức soundness hoặc completeness của toàn chương trình. Việc đạt đủ 10 lá giúp kiểm tra cấu trúc mô hình hiện tại, nhưng không thay thế kiểm tra các đường đi bị bộ đo gộp lại.

5.3. Những giới hạn của thực nghiệm

Chỉ chạy bộ ca cố định và một số ca biên bổ sung, chưa có chiến dịch fuzzing dài với corpus biến đổi tự động.

Chưa thực hiện kiểm thử tích hợp frontend, API và MySQL; không có số liệu tải, thông lượng hay mức sẵn sàng của ứng dụng.

Sanitizer chỉ quan sát các đường đi đã thực thi. Không có cảnh báo trong một lần chạy không chứng minh chương trình an toàn với mọi đầu vào.

Việc tắt phát hiện rò rỉ làm cho thực nghiệm không đánh giá memory leak.

Kích thước cấu trúc và mã kết thúc tiến trình phụ thuộc môi trường; cần ghi lại cấu hình khi tái lập trên hệ khác.

Trạng thái registry của bộ ca gốc được giữ giữa các ca, do đó kết quả phụ thuộc vào thứ tự và không tương đương 11 ca hoàn toàn cô lập.

Chưa kiểm tra tình huống malloc thất bại. Cả cấp phát ban đầu lẫn cấp phát lại đều gọi memset ngay sau malloc mà không kiểm tra kết quả, nên đây là hướng cần khảo sát tiếp.

Không áp dụng softsec-toolkit, SMT/BMC hoặc symbolic execution. Không đưa tên các kỹ thuật này vào phần thành tích như thể đã sử dụng.

Các đoạn mã khắc phục ở mục tiếp theo là đề xuất, chưa được áp dụng vào mã nguồn gốc hoặc kiểm thử hồi quy trong thực nghiệm này.

5.4. Đề xuất khắc phục lỗi và hoàn thiện bộ kiểm thử

Thứ nhất, kiểm tra đủ hai cận trước khi sao chép. Cần kiểm tra dấu trước khi đổi sang size_t; đồng thời kiểm tra kích thước nguồn và đích. Đoạn mã minh họa đề xuất:

```
int requested = tc->step5_audit_claim_len;
if (requested < 0) {
    /* Ghi nhan dau vao khong hop le. */
    return;
}
size_t length = (size_t)requested;
if (length > sizeof(old_bio_buf) || length > sizeof(diff_preview)) {
    /* Tu choi thay vi sao chep. */
    return;
}
memcpy(diff_preview, old_bio_buf, length);
```

Nếu dữ liệu đầu ra được sử dụng như chuỗi C, cần quy định thêm việc bảo đảm ký tự kết thúc. Kiểm tra kích thước sao chép chỉ xử lý an toàn vùng nhớ, chưa tự bảo đảm mọi yêu cầu ngữ nghĩa của chuỗi.

Thứ hai, xử lý kích thước bằng kiểu phù hợp. Nếu mục tiêu là tính số byte để cấp phát, nên dùng size_t và kiểm tra count > SIZE_MAX / sizeof(uint32_t) trước phép nhân. Nếu mô hình cố ý giới hạn kết quả ở 16 bit, cần kiểm tra so với UINT16_MAX trên giá trị đầy đủ trước khi ép kiểu. Thêm các giá trị biên như 16383, 16384 và 16385 vào bộ kiểm thử hồi quy.

Thứ ba, sửa cơ chế đánh giá kết quả. Tách nhãn tình huống mô phỏng khỏi lỗi thực; có thể dùng tên SIMULATED_RISK hoặc INVALID_INPUT. Chỉ thống kê crash thực khi tiến trình hoặc sanitizer cung cấp bằng chứng phù hợp. Sử dụng oracle độc lập để kiểm tra thuộc tính, thay vì chỉ dựa vào tên trạng thái được chính hàm mục tiêu ghi ra.

Thứ tư, chuẩn hóa đặc tả và số liệu. Điều chỉnh P03, P04, P07, P09, P10, P12 và P15 cho khớp yêu cầu thực sự; đồng thời cập nhật điều kiện trong code. Thay kích thước ghi cứng bằng thông tin đo được. Phần tổng kết mới đã dùng count_visited để tính các tử số; riêng tiêu đề bảng path còn ghi cố định "10/10" và nên chuyển thành giá trị tính toán. Bộ nhận dạng path cần so sánh cả độ dài lẫn toàn bộ dãy trạng thái, đồng thời phân biệt trace không thuộc đặc tả.

Thứ năm, kiểm tra quyền theo bản ghi. Với chức năng thành viên, truy vấn của người dùng thông thường cần gắn đồng thời id và clanId được suy ra từ ngữ cảnh xác thực. Không lấy phạm vi dòng họ do client tự khai báo làm căn cứ cấp quyền. Quyền quản trị liên dòng họ, nếu có, cần được định nghĩa riêng. Kiểm thử hồi quy phải có cặp tài khoản khác dòng họ và kiểm tra cả GET, PUT, DELETE.

Thứ sáu, sửa quy trình khởi tạo schema. Không loại cả câu SQL chỉ vì có chú thích mở đầu. Nên quản lý migration bằng các câu lệnh có cấu trúc hoặc công cụ migration, xử lý thứ tự tạo bảng và kiểm tra lại các cột mà route sử dụng. Chạy lại trên cơ sở dữ liệu trống để xác nhận quá trình cài đặt từ đầu, đồng thời kiểm tra không làm mất dữ liệu khi chạy lại trên cơ sở dữ liệu có sẵn.

5.5. Hướng phát triển

Có thể phát triển harness libFuzzer nhận Data và Size, giải mã thành cấu trúc đầu vào có giới hạn, đặt lại trạng thái toàn cục giữa các lần chạy và lưu đầu vào gây lỗi. Corpus nên bao gồm độ dài âm, ranh giới bộ đệm, chuỗi thiếu ký tự kết thúc khi phù hợp với giao diện nhận dữ liệu, chỉ số ngoài miền và đồ thị có chu trình. Các đầu vào này phải được đưa vào qua bộ giải mã an toàn để phân biệt lỗi harness với lỗi mục tiêu.

Ở ứng dụng web, cần bổ sung bộ kiểm thử tích hợp dùng dữ liệu giả, kiểm tra phạm vi dòng họ, cập nhật quan hệ gia đình và quy trình khởi tạo. Mọi phép so sánh giữa kỹ thuật kiểm thử trong tương lai nên dùng cùng mục tiêu, ngân sách và tập lỗi tham chiếu. Báo cáo hiện tại chưa có cơ sở để so sánh định lượng black-box và white-box fuzzing.

6. Kết luận

Báo cáo đã phân tích phiên bản cập nhật của hệ thống quản lý dòng họ và mô hình C gồm một module xử lý năm bước. Chiến dịch gồm 11 ca, đạt 16/16 nhãn trạng thái, 16/16 nhãn mệnh đề và 10/10 path khai báo. Đối chiếu độc lập xác nhận 11 trace baseline khớp chuỗi đặc tả tương ứng. Cấu trúc kết thúc sớm ở nhánh bất thường giúp thể hiện rõ điểm dừng của từng ca.

Tám nhãn "CRASH" và một nhãn "HANG" vẫn là các sự kiện do chương trình tự phân loại; baseline chạy cùng sanitizer kết thúc với mã 0. Kiểm thử bổ sung xác nhận lỗi độ dài audit âm còn tồn tại, điều

kiện phát hiện thu hẹp số vẫn bỏ sót count 16384 và một số mệnh đề không khớp điều kiện thực thi. Phép đo mới ghi nhận sizeof(ClanMember) = 72 byte trên ARM64.

Bộ đo path chỉ so khớp trạng thái cuối, vì vậy cần bổ sung so sánh đầy đủ trace để bảo đảm tính đúng khi mô hình thay đổi. Mã web không thay đổi; nguy cơ thiếu ràng buộc dòng họ và lỗi lọc SQL trong runSchema vẫn cần xử lý theo mức bằng chứng đã nêu. Hướng hoàn thiện ưu tiên kiểm tra cận đầu vào, sửa oracle và đặc tả, xác thực path, bổ sung kiểm thử tích hợp và bảo đảm quy trình dựng cơ sở dữ liệu.

Tài liệu tham khảo

[1] Vũ Quang Dũng. Hướng dẫn Báo cáo Bài tập lớn – Cấu trúc, định dạng và tiêu chí đánh giá. Học phần CSE703093 – An toàn phần mềm, Khoa Hệ thống thông tin, Trường Đại học Phenikaa. Tệp BTL_02_HuongDan_BaoCao.pdf, 27 trang, do người thực hiện cung cấp.

[2] Kho mã nguồn software_safety-main-2. Các tệp được đối chiếu chính: clan_genealogy_fuzzer.c; quanly-dongho/src/App.tsx; src/services/api.ts; src/context/AppContext.tsx; src/pages/GiaPhaPage.tsx; backend/src/routes/members.js; backend/src/middleware/auth.js; backend/src/db/schema.js; backend/src/db/pool.js; các tệp package.json, README và bảng Bao_Cao_Kiem_Thu_Fuzzing_ MocBan.xlsx gồm bốn sheet. Bản cục bộ được phân tích lại ngày 23/10/2026.

[3] LLVM Project. AddressSanitizer. https://clang.llvm.org/docs/AddressSanitizer.html. Truy cập ngày 22/10/2026.

[4] LLVM Project. UndefinedBehaviorSanitizer. https://clang.llvm.org/docs/UndefinedBehaviorSanitizer.html. Truy cập ngày 22/10/2026.

[5] LLVM Project. libFuzzer – a library for coverage-guided fuzz testing. https://llvm.org/docs/LibFuzzer.html. Truy cập ngày 22/10/2026.

[6] MITRE. CWE-195: Signed to Unsigned Conversion Error. https://cwe.mitre.org/data/definitions/195.html. Truy cập ngày 22/10/2026.

[7] MITRE. CWE-197: Numeric Truncation Error. https://cwe.mitre.org/data/definitions/197.html. Truy cập ngày 22/10/2026.

[8] MITRE. CWE-835: Loop with Unreachable Exit Condition. https://cwe.mitre.org/data/definitions/835.html. Truy cập ngày 22/10/2026.

[9] MITRE. CWE-639: Authorization Bypass Through User-Controlled Key. https://cwe.mitre.org/data/definitions/639.html. Truy cập ngày 22/10/2026.

[10] OpenAI Codex/ChatGPT. Hỗ trợ đọc và đối chiếu mã nguồn, tạo chương trình kiểm thử bổ sung, thực thi kiểm tra và biên soạn nội dung báo cáo trong các phiên làm việc ngày 22–23/10/2026. Các thao tác kiểm thử được lưu trong mã và log kèm theo.

Phụ lục

Phụ lục A. Tổ chức mã nguồn và bằng chứng

```
BTL_AnToanPhanMem_CapNhat/
  Bao_cao_An_toan_phan_mem_cap_nhat.docx
  README.md
  source/
    clan_genealogy_fuzzer.c
    README_goc.md
    quanly-dongho/
  tools/
    probe.c
    reproduce.py
    validate_traces.py
    check_schema.cjs
  results/
    baseline.stdout.txt
    baseline.stderr.txt
    audit_-1.stderr.txt
    path_mismatch.stdout.txt
    trace_validation.json
    schema_unit.txt
    manifest.json
    source_inventory.json
    workbook_reference.json
```

Mã nguồn được sao chép từ software_safety-main-2 để chạy kiểm tra; không chỉnh sửa thư mục gốc. Các công cụ trong tools/ là phần bổ sung phục vụ kiểm chứng báo cáo. Dấu SHA-256 của tệp C được dùng:

```
8bebc57d194851ceaa0cc49adccad1591e7a46c7e9f7d15bd43f5e62411b53ee
```

Phụ lục B. Chạy lại thực nghiệm

Điều kiện: Python 3, Node.js và Clang có ASan/UBSan. Các kiểm tra đã thực hiện không cần cơ sở dữ liệu MySQL. Từ thư mục gốc của gói:

```
python3 tools/reproduce.py
python3 tools/validate_traces.py
node tools/check_schema.cjs > results/schema_unit.txt
```

Script đầu biên dịch, chạy baseline và các probe, lưu stdout/stderr cùng mã kết thúc. Script thứ hai so sánh toàn bộ 11 trace baseline với 10 chuỗi path trong source. Kiểm thử schema dùng kết nối giả và ghi số câu lệnh chính được gửi. Chạy riêng hai tình huống quan trọng sau khi biên dịch:

```
ASAN_OPTIONS=detect_leaks=0 results/probe audit -1
ASAN_OPTIONS=detect_leaks=0 results/probe path_mismatch
```

Lệnh audit dự kiến bị ASan chấm dứt. Lệnh path_mismatch là kiểm thử đơn vị bộ đo, chủ động tạo trace không đầy đủ nhưng cùng trạng thái cuối PATH_01. Baseline có thể được lưu riêng bằng chuyển hướng stdout; chương trình C không tự tạo tệp log:

```
ASAN_OPTIONS=detect_leaks=0 results/baseline > results/baseline.stdout.txt
```

Ứng dụng web vẫn chưa được chạy tích hợp. Backend đọc các biến DB_HOST, DB_PORT, DB_USER, DB_PASSWORD, DB_NAME, JWT_SECRET; frontend dùng VITE_API_URL, mặc định http://localhost:3001/api. Trước khi dựng trên cơ sở dữ liệu mới cần xử lý lỗi schema ở mục 4.7. Hướng dẫn npm trong README không được dùng làm bằng chứng ứng dụng đã chạy thành công.

Phụ lục C. Đối chiếu kết luận với bằng chứng

Nội dung kiểm tra	Tệp bằng chứng
11 ca, ba tỷ lệ độ phủ và các nhãn sự kiện	results/baseline.stdout.txt
Baseline kết thúc 0, stderr không có chẩn đoán	results/manifest.json, results/baseline.stderr.txt
Lỗi audit âm	results/audit_-1.stderr.txt
Các biên audit và fund	Các tệp results/audit_*.stdout.txt, results/fund_*.stdout.txt
Sai lệch mệnh đề	results/bio.stdout.txt, results/spouse_ 14.stdout.txt, results/lifecycle_3.stdout.txt

Kích thước cấu trúc 72 byte	results/size.stdout.txt
Bộ nhận dạng chỉ xét trạng thái cuối	results/path_mismatch.stdout.txt
11 trace thực tế khớp toàn bộ chuỗi khai báo	results/trace_validation.json
Lỗi khởi tạo bảng chính	results/schema_unit.txt
Quy mô và dấu kiểm tra từng tệp nguồn	results/source_inventory.json
Nội dung bốn sheet tài liệu tham khảo	results/workbook_reference.json